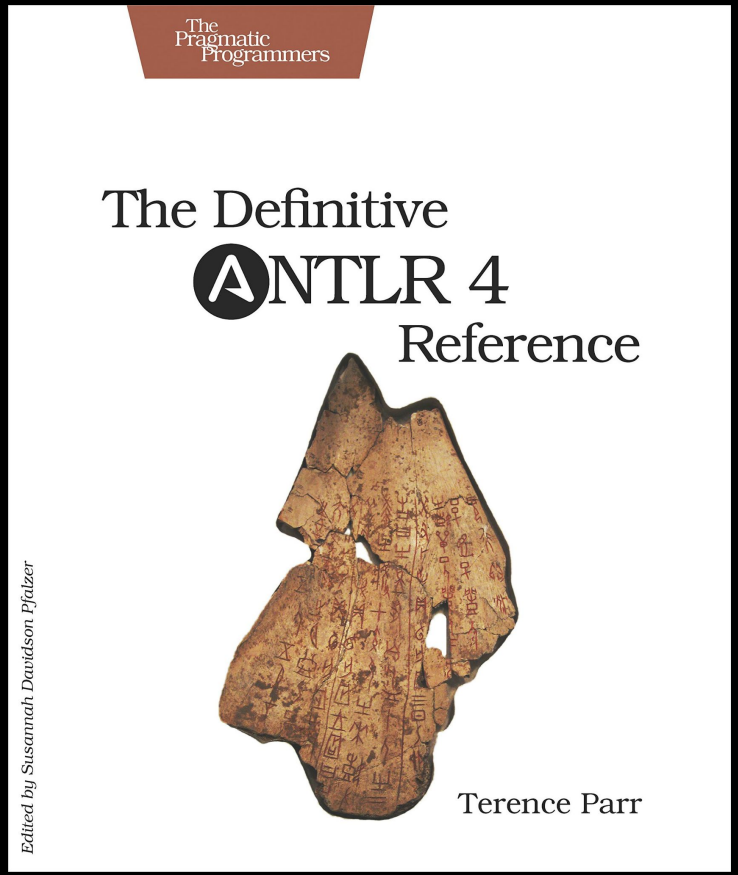
ANTLR - A Compiler for Compilers

Abhishek D

What is ANTLR



Why ANTLR

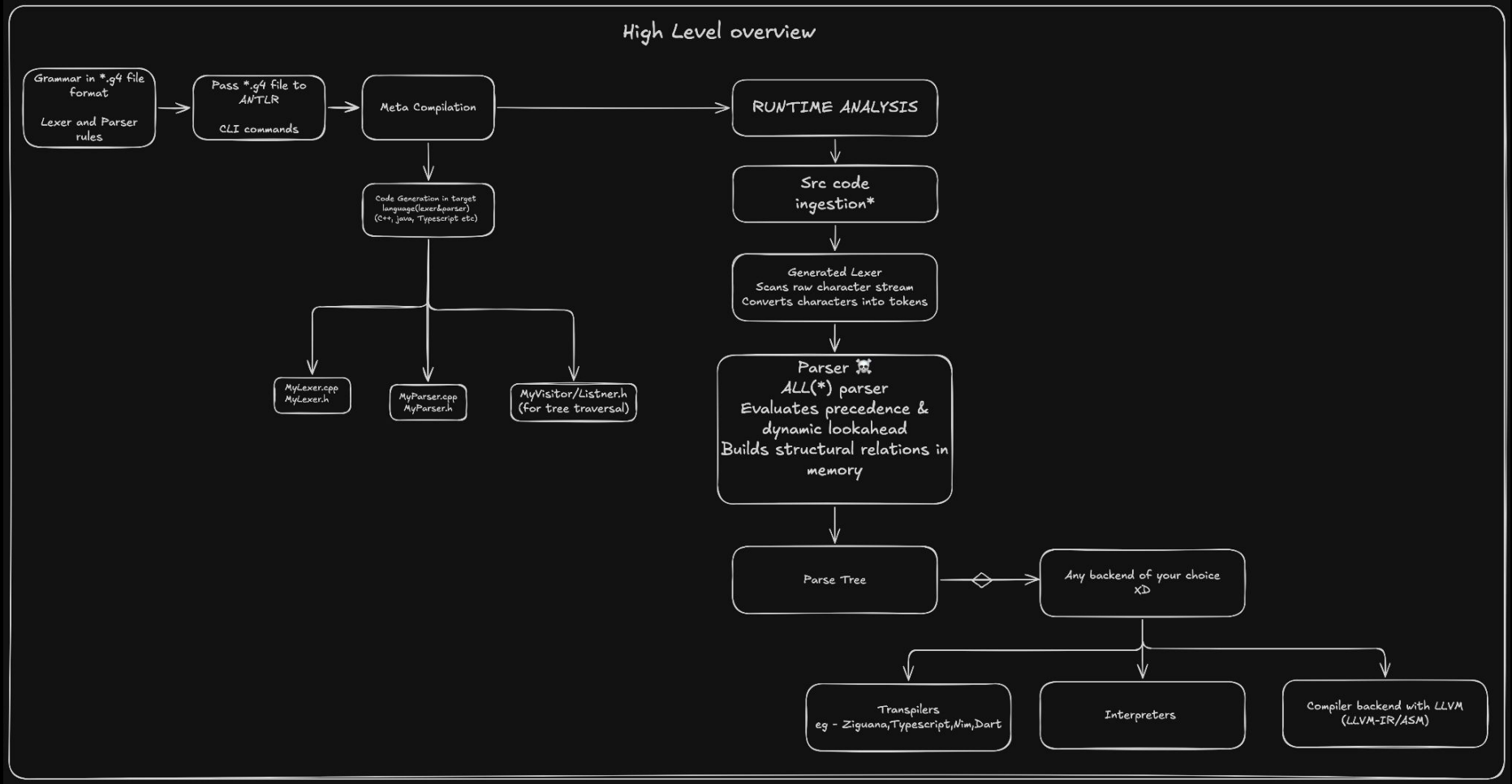
Hard to maintain parsers as the language grows



Faster

Easier ?

Multi-Language Code Generation


High Level overview
Grammar in *.g4 file format
Lexer and Parser rules
Pass *.g4 file to ANTLR
CLI commands
Meta Compilation
Code Generation in target language(lexer&parser)
(C++, Java, Typescript etc)
MyLexer.cpp
MyLexer.h
MyParser.cpp
MyParser.h
MyVisitor/Listner.h
(for tree traversal)
RUNTIME ANALYSIS
Src code ingestion*
Generated Lexer
Scans raw character stream
Converts characters into tokens
Parser ☠
ALL(*) parser
Evaluates precedence & dynamic lookahead
Builds structural relations in memory
Parse Tree
Any backend of your choice
xD
Transpilers
eg - Ziguana,Typescript,Nim,Dart
Interpreters
Compiler backend with LLVM
(LLVM-IR/ASM)

A Bit of CFG

Definition - Set of production rules used to define a set of strings

$$G = (V, \Sigma, R, S)$$

A Bit of CFG

V -> variables/Non-Terminals -> Grammar to be expanded

Σ -> Terminals ->Concrete tokens/characters

P ->Rules/Production ->a set of recipe-like instructions

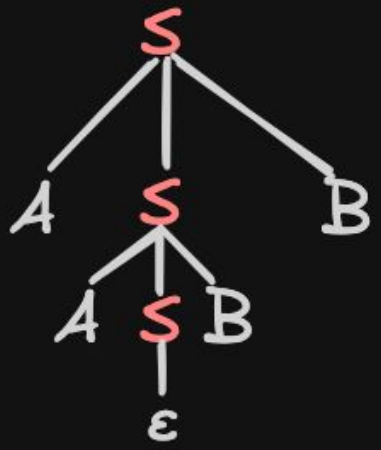
S -> Root->derivation begins here

A Bit of CFG

$L = \{a^n b^n \mid n >= 0\}$

S -> aSb | ε

derive aabb



S=aSb

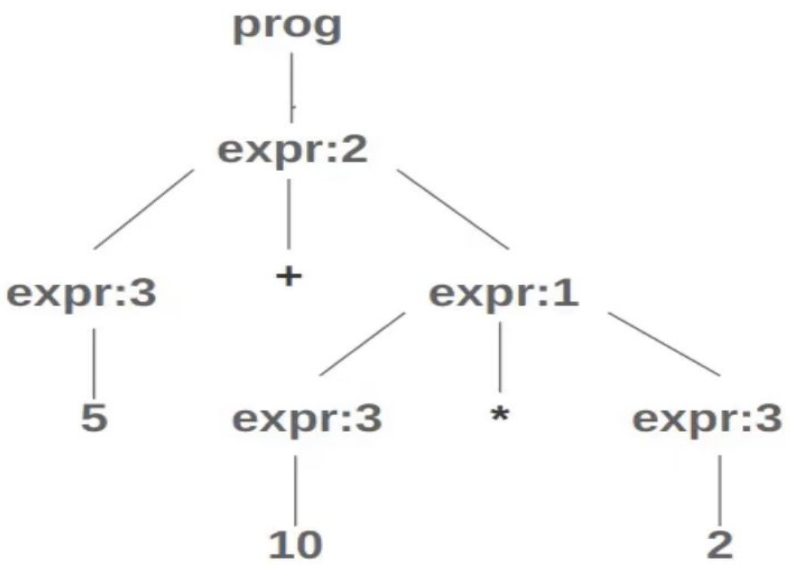
S=aSb ⟶ S=a(aSb)b ⟶ S=aabb

ε

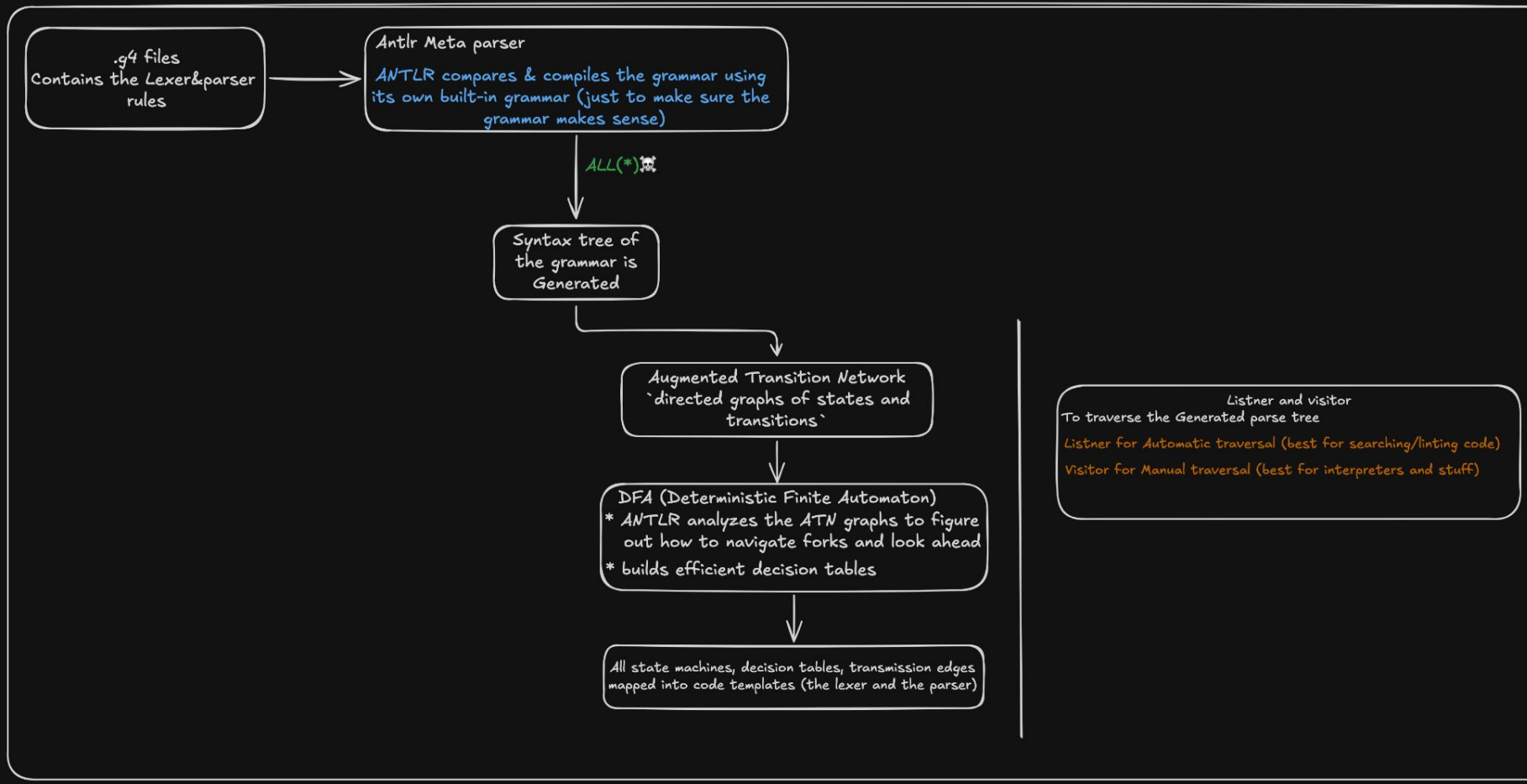
But how is this even about compilers ?

```
grammar Calculator;
prog:   (expr)* ;
expr:   expr ('*'|'/') expr
    |   expr ('+'|'-') expr
    |   INT
    |   '(' expr ')'
    ;
INT     : [0-9]+ ;
```

5 + 10 * 2 = ?



Teach me ANTLR Like I am 5


.g4 files
Contains the Lexer&parser rules
Antlr Meta parser
ANTLR compares & compiles the grammar using its own built-in grammar (just to make sure the grammar makes sense)
ALL(*)☠
Syntax tree of the grammar is Generated
Augmented Transition Network `directed graphs of states and transitions`
DFA (Deterministic Finite Automaton)
* ANTLR analyzes the ATN graphs to figure out how to navigate forks and look ahead
* builds efficient decision tables
All state machines, decision tables, transmission edges mapped into code templates (the lexer and the parser)
Listner and visitor
To traverse the Generated parse tree
Listner for Automatic traversal (best for searching/linting code)
Visitor for Manual traversal (best for interpreters and stuff)

ALL(*)

Generated parser is an ALL(*) parser

Ambiguous grammar has multiple choices that can be taken

The algorithm is able to Dynamic ` Lookahead ` (automatic sub-analysis that looks ahead as many tokens as necessary—even to the end of the file)

ALL(*) records its decision-making paths in a **Deterministic Finite Automaton (DFA) cache** at runtime

What ANTLR can do and what is cant

What it can do

Understand ambiguous grammar

Compiler frontend generation

Validate Structured Data

What it can't

No semantic analysis

Not the most optimised (slower compared to heavily optimised production ready parsers)

```
grammar Calculator;

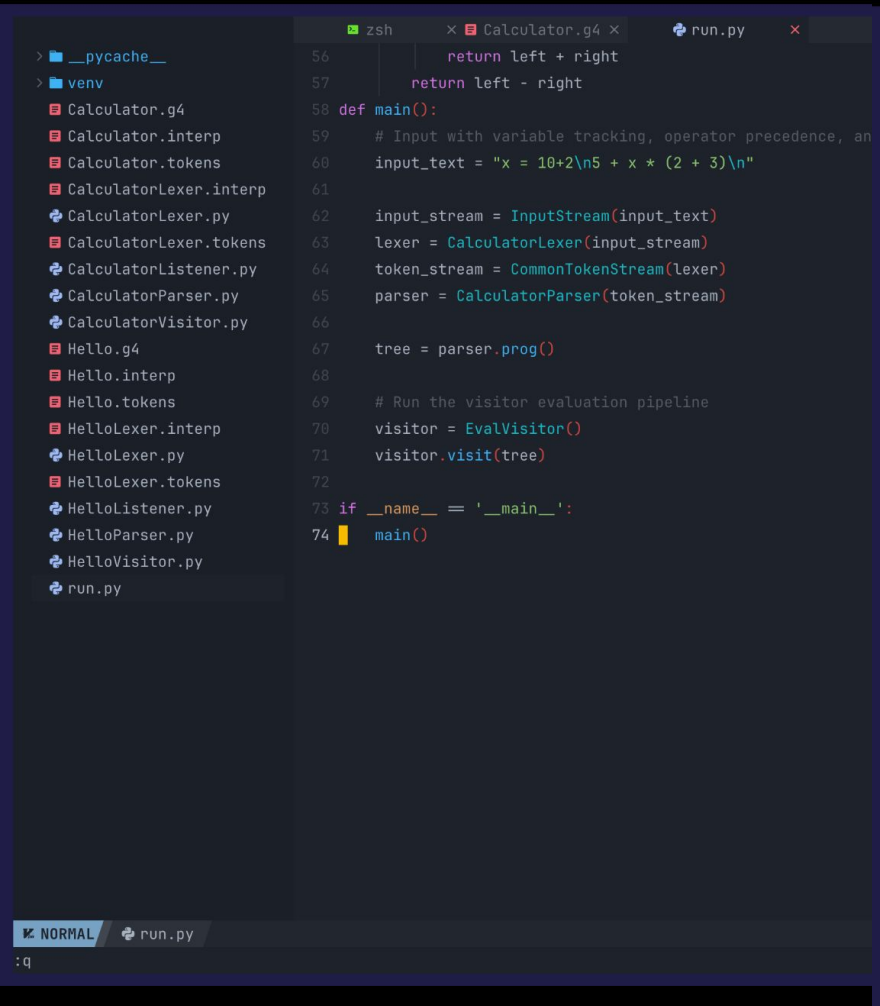
// Entry point rule: allows multiple expressions separated by newlines or semicolons
prog
    : stat+ EOF
    ;

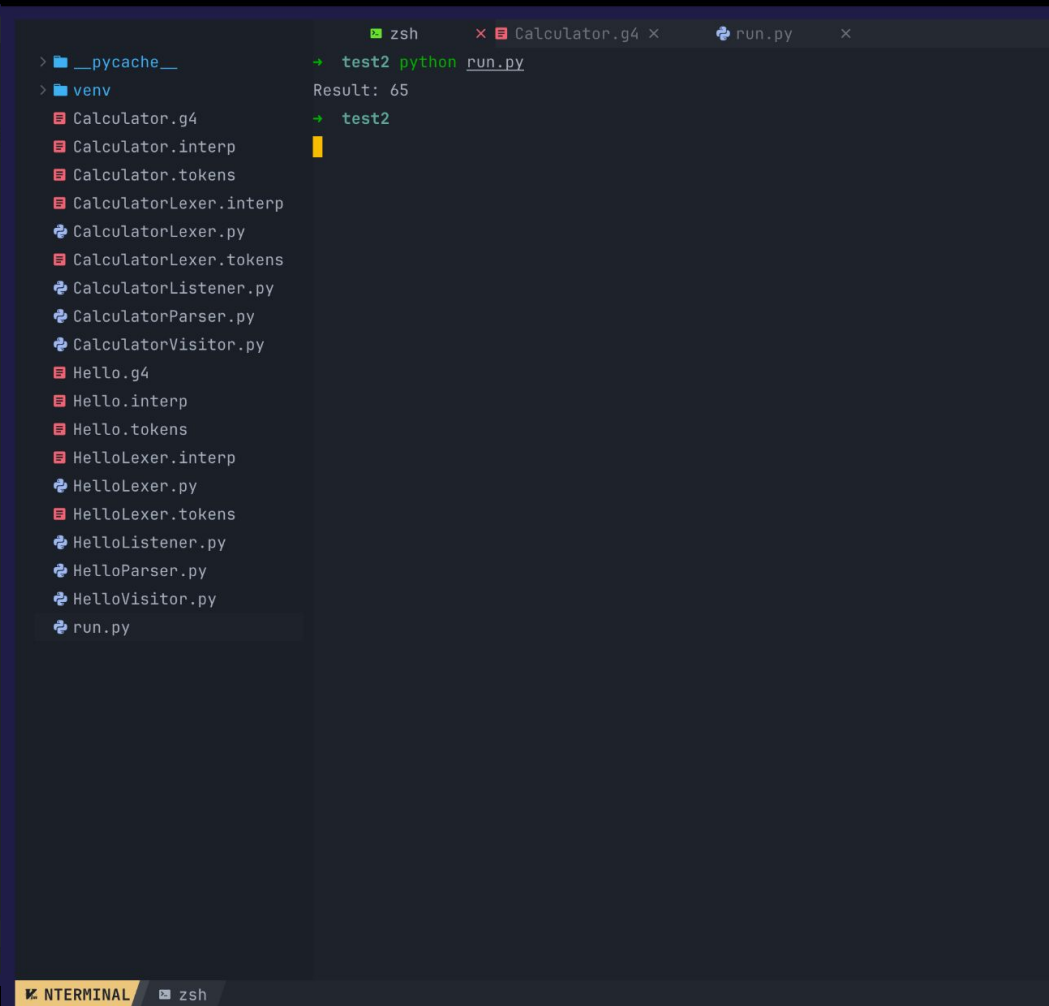
stat
    : expr NEWLINE                # printExpr
    | ID '=' expr NEWLINE         # assign
    | NEWLINE                     # blank
    ;

expr
    : expr op=('*'|'/') expr      # MulDiv

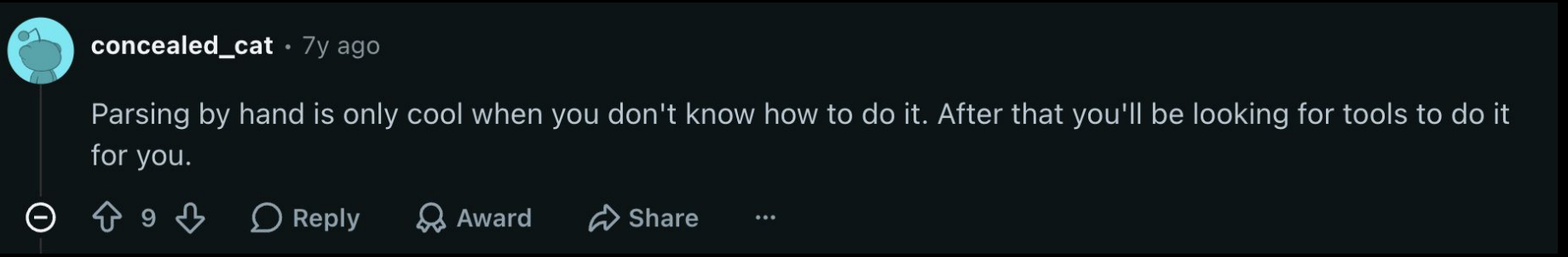
    | expr op=('+'|'-') expr      # AddSub
    | INT                         # int
    | ID                          # id
    | '(' expr ')'                # parens
    ;

// Lexer Rules
ID      : [a-zA-Z]+ ;
INT     : [0-9]+ ;
NEWLINE : [\r\n]+ ;
WS      : [ \t]+ -> skip ;
```


zsh
Calculator.g4
run.py
__pycache__
venv
Calculator.g4
Calculator.interp
Calculator.tokens
CalculatorLexer.interp
CalculatorLexer.py
CalculatorLexer.tokens
CalculatorListener.py
CalculatorParser.py
CalculatorVisitor.py
Hello.g4
Hello.interp
Hello.tokens
HelloLexer.interp
HelloLexer.py
HelloLexer.tokens
HelloListener.py
HelloParser.py
HelloVisitor.py
run.py
return left + right
return left - right
def main():
Input with variable tracking, operator precedence, an
input_text = "x = 10+2\n5 + x * (2 + 3)\n"
input_stream = InputStream(input_text)
lexer = CalculatorLexer(input_stream)
token_stream = CommonTokenStream(lexer)
parser = CalculatorParser(token_stream)
tree = parser.prog()
Run the visitor evaluation pipeline
visitor = EvalVisitor()
visitor.visit(tree)
if __name__ == '__main__':
main()
NORMAL
run.py
:q


zsh
Calculator.g4
run.py
__pycache__
venv
Calculator.g4
Calculator.interp
Calculator.tokens
CalculatorLexer.interp
CalculatorLexer.py
CalculatorLexer.tokens
CalculatorListener.py
CalculatorParser.py
CalculatorVisitor.py
Hello.g4
Hello.interp
Hello.tokens
HelloLexer.interp
HelloLexer.py
HelloLexer.tokens
HelloListener.py
HelloParser.py
HelloVisitor.py
run.py
test2 python run.py
Result: 65
test2
NTERMINAL
zsh
:q

Are Hand Written Parsers dead ?



Thank You